Week 2: Convolutional Networks and the UNet

Theme:	Build the architecture that powers every diffusion model
Time Commitment:	8–12 hours
Suggested Split:	3h reading/video, 7h coding (UNet is dense), 1h review

Learning Objectives

By the end of this week, you should be able to:

- Explain how convolutions, pooling, and feature maps work
- Implement skip connections and articulate why they matter
- Build a UNet from scratch with encoder, bottleneck, and decoder blocks
- Train a UNet as a denoising autoencoder on image data

Primary Resources (Must-Do)

[VIDEO] Stanford CS231n Lecture 5: Convolutional Neural Networks

Type: Lecture Video | **Time:** 75 min | **Difficulty:** Beginner-Intermediate
URL: https://www.youtube.com/watch?v=bNb2fEVKeEo

Why this resource: The clearest visual explanation of convolutions, strides, padding, and pooling that exists. Justin Johnson's diagrams have been copied into hundreds of textbooks.

Focus on: Convolution mechanics, output dimension calculations, and the historical progression from LeNet to ResNet.

[PAPER] U-Net: Convolutional Networks for Biomedical Image Segmentation (Ronneberger et al., 2015)

Type: Research Paper | **Time:** 45–60 min | **Difficulty:** Intermediate
URL: https://arxiv.org/html/1505.04597v1

Why this resource: The original UNet paper. Surprisingly readable. The architecture diagram (Figure 1) is iconic — you'll see it referenced everywhere in the diffusion world.

Focus on: Sections 2 (Network Architecture) and 3 (Training). Skip the medical imaging specifics.

[BLOG/DOCS] A Guide to Convolution Arithmetic for Deep Learning (Dumoulin & Visin)

Type: Tutorial Paper | **Time:** 30–45 min | **Difficulty:** Intermediate
URL: https://arxiv.org/pdf/1603.07285

Why this resource: The definitive reference for understanding output dimensions in convolutional layers. Saves hours of off-by-one debugging.

Focus on: The animated GIFs in the companion repo (Go through read me) : https://github.com/vdumoulin/conv_arithmetic

[BLOG/DOCS] Aman Arora's UNet Blog Post

Type: Blog Post + Code | **Time:** 45 min | **Difficulty:** Intermediate
URL: https://amaarora.github.io/posts/2020-09-13-unet.html

Why this resource: Walks through implementing UNet in PyTorch line-by-line with clear explanations of each block. Excellent companion while you code.

Focus on: The implementation sections — type the code yourself rather than copy-pasting.

This Week's Deliverable

Implement a UNet from scratch in PyTorch and train it as a denoising autoencoder. Workflow:

1. Take MNIST or CIFAR-10 images
2. Add Gaussian noise of varying magnitudes
3. Train the UNet to reconstruct the clean images

Your code must include:

- Modular `DoubleConv`, `Down`, and `Up` blocks (separate classes)
- Configurable depth and channel count
- Skip connections that handle dimension mismatches gracefully
- Visualization of (noisy input, denoised output, ground truth) triplets every few epochs

This UNet will be the backbone of your diffusion model in Week 4 — build it well.

Key Concepts Checklist

You should be able to confidently answer all of the following:

- Why are skip connections in a UNet important? What happens without them?
- How do you compute the output dimensions of a Conv2d layer?
- What's the difference between max pooling and strided convolutions for downsampling?
- Why use transposed convolutions vs. bilinear upsampling for the decoder?
- How would you modify your UNet to handle 3-channel (RGB) images instead of grayscale?
- Why does UNet work so well for image-to-image tasks specifically?

Stretch Resources (Optional)

- **Stanford CS231n full notes:** https://cs231n.github.io/convolutional-networks/
- **ResNet paper** (He et al., 2015): https://arxiv.org/pdf/1512.03385 — helps understand why residual/skip connections work
- **Add attention to your UNet:** Read about self-attention blocks in transformers and add one at the bottleneck
- **Compare denoising performance:** UNet vs. a vanilla CNN encoder-decoder without skips

Debugging & Reference

- **Common UNet bugs:** Skip connection dimension mismatches — print shapes liberally
- **Reference implementation (read, don't copy):** https://github.com/milesial/Pytorch-UNet
- **PyTorch CNN tutorial:** https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html